

High Performance Computing - Assignment 5

Areeb Qureshi

January 2026

1 Serial Implementation and Validation

1.1 Overview of the Serial Code

The starting point for this project was the provided serial implementation of a gravitational N -body simulation modelling the motion of stars in the gravitational potential of a central black hole. The equations of motion are integrated using the Velocity-Verlet (VV) algorithm, which is time-reversible and exhibits good long-term energy conservation properties for Hamiltonian systems.

The data structures and initialisation routines are supplied in `data.c`. In particular, the function `initialiseCluster()` generates the initial stellar masses, positions, and velocities based on the specified cluster parameters.

1.2 Velocity-Verlet Integration

The velocity Verlet algorithm advances the system through the following steps at each timestep:

1. Velocities are advanced by half a timestep using the current acceleration.
2. Positions are advanced by a full timestep using the updated velocities.
3. Accelerations are recomputed from the updated positions.
4. Velocities are advanced by the remaining half timestep.

This scheme was implemented explicitly using the equations provided in the assignment. A timestep of 1×10^{-5} was chosen to ensure numerical stability while allowing the system to evolve over many timesteps without significant energy drift. See Figure 1 that shows the complete orbit of a single star post-implementation of VV algorithm.

1.3 Inclusion of Star–Star Interactions

Initially, only the gravitational acceleration due to the central black hole was included (see Figure 2 for which the following initial conditions were used (-M 150000 $M_{\odot}$, -S 20000)). The second term in the force equation was then

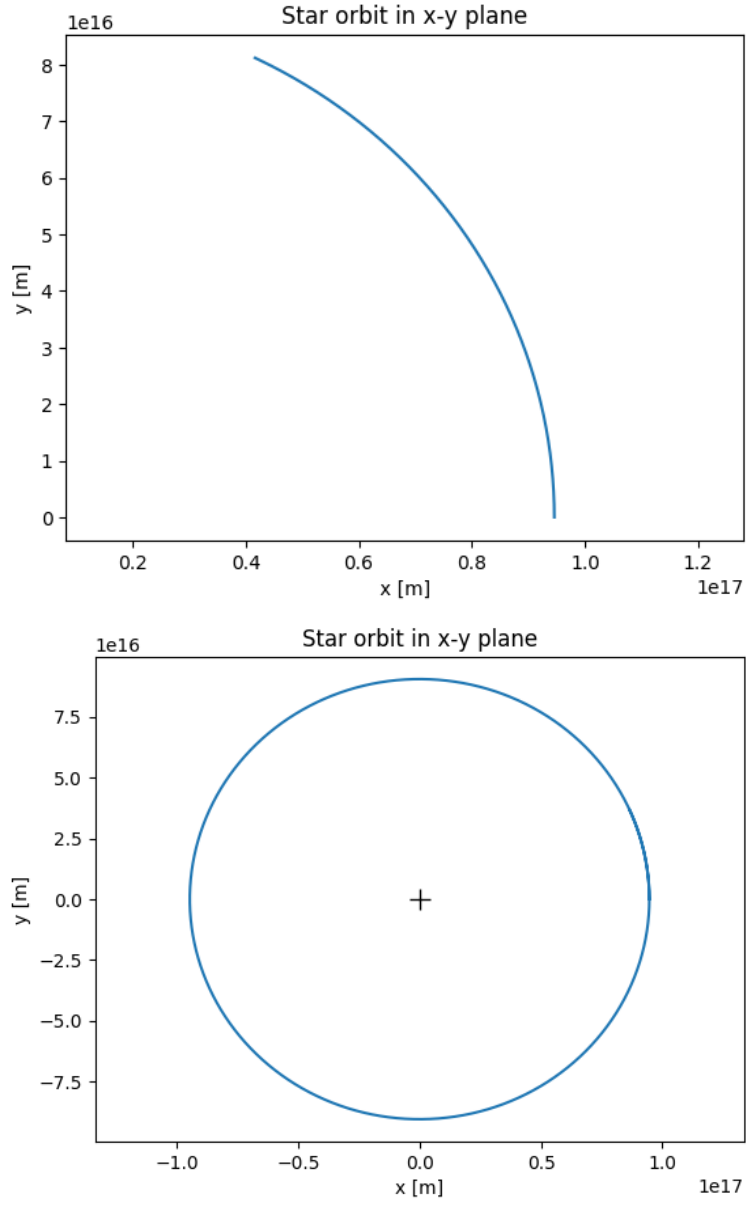


Figure 1: *Top*: Shows an incomplete orbit for 1 star post-implementation of VV algorithm. The number of steps were adjusted to 120000 to produce *bottom*: a full orbit of 1 star.

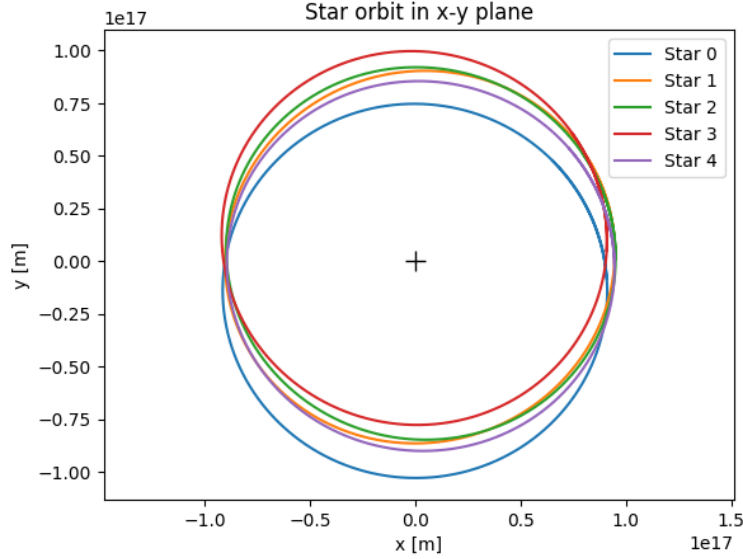


Figure 2: Orbit of 5 stars without any interactions

implemented to account for pairwise gravitational interactions between stars. This introduces an $O(N^2)$ computational cost per timestep and dominates the runtime for sufficiently large N .

Care was taken to avoid self-interactions, handle small separations safely, and accumulate forces correctly for each star. At this stage, the serial code models a self-gravitating stellar cluster in the vicinity of a massive black hole. It must be noted that the star-star interactions were not dominant using default settings. After making the cluster mass comparable to that of the black hole, the star-star interactions became very noticeable as seen in Figure 3.

1.4 Bookkeeping and Terminal Events

Each star is monitored for two terminal conditions:

- **Swallowing**, when a star crosses the black hole event horizon,

$$r + R_{\star} < R_{\text{BH}}, \quad (1)$$

- **Escape**, when a star exceeds a predefined escape radius,

$$r > R_{\text{escape}}. \quad (2)$$

When either condition is met, the star is marked inactive and removed from further integration. Although it is difficult in practice to tune parameters such that stars naturally cross the event horizon, the logic for detecting and handling

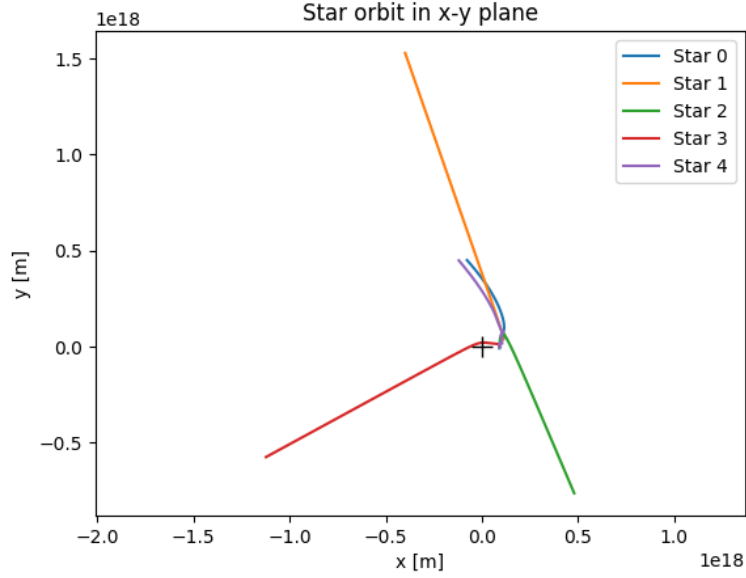


Figure 3: Orbit of 5 stars after including star-star interactions

both events was fully implemented. It was ensured that stars marked as inactive no longer had their positions updated, however, their mass and positions are still being used when computing the gravitational acceleration of all the other stars. This was done by NOT adding the following line of code to the acceleration loop: `if (active[jstar] == 0) continue;`

1.5 Serial Sanity Checks

A number of sanity checks were performed on the serial implementation:

- Stable orbital behaviour for sufficiently small timesteps.
- Reasonable qualitative evolution of stellar trajectories.
- Correct deactivation of stars upon escape or swallowing.
- Consistent reporting of the nearest-star distance to the black hole.

These checks confirm that the serial implementation correctly follows the intended physical and numerical model.

2 Parallel Implementation

2.1 Timing and Profiling of the Serial Code

As a first step towards optimisation and parallelisation, the serial implementation was instrumented with timers in order to accurately determine where computational time is spent. Timers were added around the main stages of the time-stepping loop using `omp_get_wtime()`, allowing the relative importance of each component to be quantified. This profiling was performed for a range of system sizes in order to investigate how the distribution of runtime changes as the number of stars N increases.

The timed components include the velocity–Verlet updates (T_{vv} and T_{vv2}), the gravitational acceleration calculation (T_{acc}), the dust drag contribution (T_{drag}), bookkeeping operations such as escape and event-horizon checks (T_{book}), diagnostic output (T_{output}), and communication-related overheads ($T_{comm,pos}$ and $T_{comm,misc}$). The resulting timing breakdown is summarised in Table 1.

The results clearly demonstrate that the gravitational acceleration calculation dominates the total runtime, particularly as the number of stars increases. For small systems ($N = 50$), the acceleration term already exceeds all other components, and its contribution grows rapidly with increasing N . By $N = 1000$, T_{acc} is more than an order of magnitude larger than any other timed section. This behaviour is consistent with the expected $\mathcal{O}(N^2)$ scaling of the direct pairwise force calculation, in which each star interacts with every other star at each timestep.

In contrast, the velocity–Verlet position and velocity updates (T_{vv} and T_{vv2}) increase much more slowly with N , reflecting their $\mathcal{O}(N)$ computational complexity. Similarly, the drag force calculation scales linearly with N and remains a relatively small contribution to the overall runtime, even for the largest systems considered. These trends confirm that the integration scheme itself is not a performance bottleneck.

Bookkeeping operations, including checks for escape from the simulation domain or crossing of the black hole event horizon, contribute only a minor fraction of the total runtime across all system sizes. Diagnostic output is negligible by comparison, particularly as the number of stars increases. Communication-related overheads also remain small relative to the computational cost, indicating that they do not significantly affect performance in the serial implementation.

It is important to note that additional diagnostic print statements introduced during early stages of development were commented out for the purpose of timing measurements, in order to avoid artificially inflating the measured runtimes. However, all output statements present in the original supplied code were retained, ensuring that the program’s original functionality and output format remain unchanged.

Overall, this profiling analysis demonstrates that the dominant performance bottleneck arises from the gravitational acceleration calculation. The increasing dominance of T_{acc} with system size strongly motivates the focus on parallelising

and optimising this section of the code. Since the remaining components contribute comparatively little to the total runtime, further optimisation of these sections would yield limited performance gains. These results therefore provide a clear and quantitative justification for the parallelisation strategy adopted in subsequent sections.

N	$T_{\text{vv}}(s)$	$T_{\text{acc}}(s)$	$T_{\text{drag}}(s)$	$T_{\text{vv2}}(s)$
50	1.678955×10^{-3}	3.092168×10^{-2}	3.207801×10^{-3}	9.368230×10^{-4}
100	2.078412×10^{-3}	1.220135×10^{-1}	5.650440×10^{-3}	1.149921×10^{-3}
200	2.845533×10^{-3}	4.842267×10^{-1}	1.058462×10^{-2}	1.521597×10^{-3}
500	5.224315×10^{-3}	2.997378×10^0	2.517537×10^{-2}	2.681477×10^{-3}
1000	9.150103×10^{-3}	1.200773×10^1	5.003189×10^{-2}	4.642839×10^{-3}

N	$T_{\text{book}}(s)$	$T_{\text{output}}(s)$	$T_{\text{comm,pos}}(s)$	$T_{\text{comm,misc}}(s)$
50	1.057498×10^{-3}	8.763559×10^{-5}	1.090635×10^{-4}	4.020128×10^{-4}
100	1.348266×10^{-3}	8.843280×10^{-5}	1.029596×10^{-4}	4.066341×10^{-4}
200	1.897397×10^{-3}	1.102686×10^{-4}	1.225527×10^{-4}	4.985370×10^{-4}
500	3.590703×10^{-3}	1.583360×10^{-4}	1.154505×10^{-4}	6.144494×10^{-4}
1000	6.392030×10^{-3}	1.832359×10^{-4}	1.242720×10^{-4}	8.568466×10^{-4}

Table 1: Timing breakdown of major components of the simulation as a function of the number of stars N . All times are measured in seconds.

2.2 Parallelisation and Optimisation Strategy

Based on the profiling results obtained from the serial implementation, the dominant computational cost was identified as the gravitational acceleration calculation, which scales as $\mathcal{O}(N^2)$. This motivated the parallelisation strategy adopted in this work, with particular emphasis on maintaining code correctness, readability, and portability.

A hybrid MPI/OpenMP approach was selected for the parallel implementation. This choice allows the code to exploit distributed-memory parallelism via MPI while simultaneously benefiting from shared-memory parallelism through OpenMP threading.

The core time-stepping loop was parallelised by distributing subsets of stars across MPI ranks. Each MPI process is responsible for updating a contiguous block of stars, determined by a simple block decomposition. Within each process, OpenMP is used to parallelise the computationally intensive loops over stars, particularly within the velocity–Verlet update and the force calculation.

2.3 OpenMP Parallelisation

OpenMP parallelism was introduced using `#pragma omp parallel` and `#pragma omp for` directives with `default(none)` to enforce explicit variable scoping.

This approach improves code safety and clarity by ensuring that all shared and private variables are intentionally specified. Static scheduling was used throughout, as the workload per iteration is uniform and predictable.

The acceleration calculation, which dominates the runtime, was parallelised over the outer loop in which accelerations are computed for each star. Since each star’s acceleration is accumulated independently, this loop is naturally thread-safe and requires no synchronisation beyond the implicit barrier at the end of the OpenMP work-sharing construct.

2.4 MPI and Rank-0 Responsibilities

The parallel implementation makes explicit use of MPI to coordinate distributed-memory execution. The MPI environment is initialised and finalised using `MPI_Init` and `MPI_Finalize`, ensuring that all MPI functionality is correctly set up and terminated. The total number of MPI processes and each process’s rank are obtained using `MPI_Comm_size` and `MPI_Comm_rank`, respectively.

To ensure that the parallel code uses identical initial conditions to the serial implementation, all simulation initialisation is performed on rank 0 only. This includes reading command-line arguments, generating initial star properties, and seeding the random number generator. The resulting initial data arrays are then distributed to all MPI processes using `MPI_Bcast`. This approach guarantees that every MPI rank begins with an identical copy of the simulation state, while avoiding inconsistencies that could arise from multiple processes independently initialising random data.

During the time-stepping loop, each MPI process updates a subset of stars assigned via a simple block decomposition. After local position updates, global consistency is restored using collective communication. In particular, `MPI_Allgather` is used to exchange updated position arrays between all MPI ranks, ensuring that each process has access to the full set of star positions required to compute gravitational interactions with all other stars at every timestep, as required by Equation (4).

Global quantities used for diagnostics and bookkeeping are computed using MPI reduction operations. The minimum distance of any star from the black hole is obtained via `MPI_Allreduce` with the `MPI_MIN` operator, while the total number of active stars is similarly accumulated across all processes. Event information, such as star escape or encounter with the event horizon, is combined using `MPI_Reduce` to rank 0, where it can be processed in a deterministic and well-defined order.

All output to the screen and to `output.dat` is performed exclusively by rank 0. Restricting I/O operations to a single MPI process prevents race conditions and ensures that the format and ordering of the output exactly match that of the serial code. This design choice is critical for correctness, as it guarantees reproducible output independent of the number of MPI processes or OpenMP threads used.

By centralising initialisation and output on rank 0, and by using collective MPI operations to synchronise shared data, the parallel implementation pre-

serves both numerical correctness and deterministic behaviour. These measures ensure that the parallel code produces results consistent with the serial implementation while remaining scalable to larger processor counts.

2.5 Timing Methodology and Synchronisation

Accurate timing measurements were obtained using `omp_get_wtime()` and, where appropriate, MPI collective synchronisation via `MPI_Barrier`. Timers were placed around the main components of the time-stepping loop, including velocity-Verlet updates, force calculations, bookkeeping, output, and communication. This allowed the relative contributions of computation and communication to be quantified and tracked as the number of stars and processors varied.

To ensure accurate timings, additional diagnostic print statements introduced during early development were safeguarded using `#ifdef TIMING #endif` to only run while debugging (make TIMING), while all original output statements supplied with the base code were preserved. This minimised I/O interference while maintaining the expected program behaviour, as the debug statements were stored in a separate file called `timing.dat`.

2.6 Verification of Correctness

Correctness of the parallel implementation was verified by comparing the numerical output of the serial and parallel codes under identical initial conditions. The parallel code was found to reproduce the same trajectories, event detections, and diagnostic outputs as the serial implementation. For example, the nearest object distances vs steps were compared between the serial and parallel implementation using the `(diff)` command in the terminal, which produced no output, indicating that the outputs were identical. Furthermore, two plots were generated to ensure reproducibility under same initial conditions between serial and parallel implementation, as well using multiple ranks and processes, see Figure 4. The initial conditions used for this were: `-N 1000 -S 2000 -D 1e-5 -A 200 -P 2 -R 5 -M 5`.

For further verification, extreme initial conditions were used to force escape and swallow events, to ensure their reproducibility in the parallel implementation. While attempts at forcing swallow events were unsuccessful, escape events could be reproduced in both serial and parallel codes, as seen in Figure 5. It was also ensured that `output.dat` is identical between the two codes. The escape event was forced by using the following initial conditions: `-N 50 -S 50000 -D 1e-5 -M 1 -R 1e-3 -A 1e-2 -P 1e-4 -K 0.0`. It should be noted that I was expecting a swallow event for these conditions as I defined minimal kinetic energy, a small cluster and small apogee and perigee, however got an escape event. Moreover, the value of `EscapeRadius` variable in `data.c` was temporarily reduced from `200000 * Ly` to `2000 * Ly` for a different run (not plotted) to successfully force an escape event. The original value was later restored.

Overall, the chosen hybrid MPI/OpenMP parallelisation strategy successfully accelerates the dominant computational components of the simulation

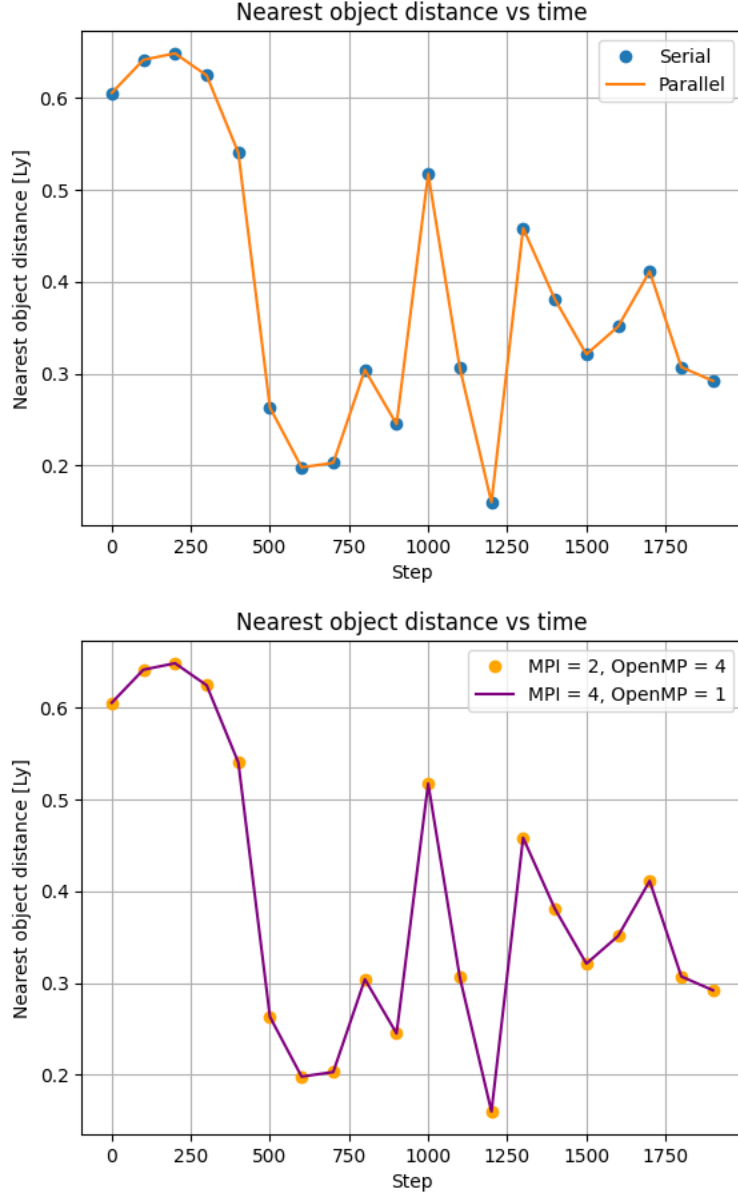


Figure 4: *Top:* Shows a plot of nearest object distance vs steps. The identical output between serial and parallel implementation can be seen. *Bottom:* Same plot but with a hybrid parallel implementation (4 MPI ranks and 1 Task vs 2 MPI ranks and 4 Tasks)

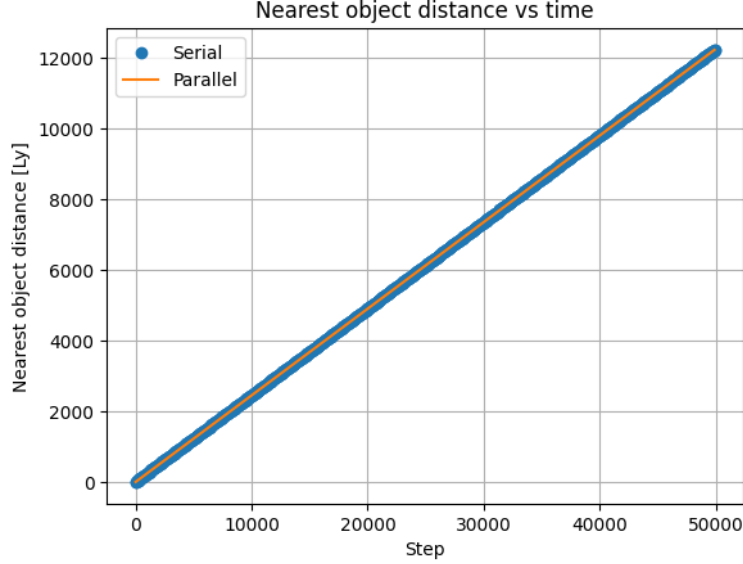


Figure 5: Serial and Parallel implementation produce reproducible results even under extreme initial conditions.

while preserving correctness and code clarity. The observed performance behaviour is consistent with expectations based on the algorithmic structure of the N -body problem, and the remaining performance limitations are primarily attributable to parallel overheads rather than un-parallelised serial regions.

2.7 Compliance with Assignment Constraints and Correctness Requirements

The parallel implementation was developed in strict accordance with the constraints specified in the assignment brief. In particular, the code is capable of running on an arbitrary number of processors, up to $P = 96$ for MPI or hybrid MPI/OpenMP configurations, subject to resource availability on the target system.

The random number generator supplied with the original code was not modified, and all initialisation routines were executed in a manner consistent with the serial version. This ensured that any differences in performance or behaviour arise solely from parallel execution rather than from changes to the underlying physics or initial conditions.

The format of all output written to the screen and to `output.dat` was preserved exactly as in the serial code. To avoid race conditions, output operations were restricted to a single MPI rank and thread, ensuring deterministic and correctly ordered output. Any additional diagnostic output introduced during development was either written to separate files or commented out for produc-

tion runs, so that the original output format is retained.

The parallel regions were designed to avoid race conditions and deadlocks. Independent per-star quantities were updated in thread-private contexts, while collective quantities were handled using appropriate MPI reduction operations. No shared writable data is accessed without proper synchronisation, and the program was tested across a range of MPI/OpenMP configurations without exhibiting non-deterministic behaviour.

Finally, while opportunities for serial optimisation were identified, aggressive restructuring of data layouts or algorithms was deliberately avoided to achieve a trade-off between absolute performance and code readability, maintainability, and portability.

3 Measuring and Reporting Performance Metrics

To evaluate the effectiveness of the chosen parallelisation strategy, performance measurements were collected on the Avon system using the final parallel implementation. Two metrics were considered: the speedup $\psi(P)$ and the Karp–Flatt metric $F(P)$, which provides an experimentally determined estimate of the serial fraction of the code.

The speedup is defined as

$$\psi(P) = \frac{T(1)}{T(P)}, \quad (3)$$

where $T(1)$ is the runtime of the serial execution and $T(P)$ is the runtime using P processors, with P denoting the product of total number of MPI ranks and OpenMP threads.

The Karp–Flatt metric is given by

$$F(P) = \frac{\frac{1}{\psi(P)} - \frac{1}{P}}{1 - \frac{1}{P}}, \quad (4)$$

and provides an estimate of the fraction of the computation that is effectively serial, including both inherently serial code and parallel overheads such as communication and synchronisation.

Measurements were performed for system sizes of $N = 500, 600$, and 750 stars, and for a range of processor counts P .

3.1 Speedup and Karp–Flatt Behaviour

Figure 6 shows the measured speedup as a function of P for the three system sizes. In all cases, the speedup increases with processor count, indicating effective parallelisation of the dominant computational workload.

For larger system sizes, the achieved speedup is significantly higher. At $P = 48$, the speedup reaches approximately $\psi \approx 16$ for $N = 500$, $\psi \approx 23$ for

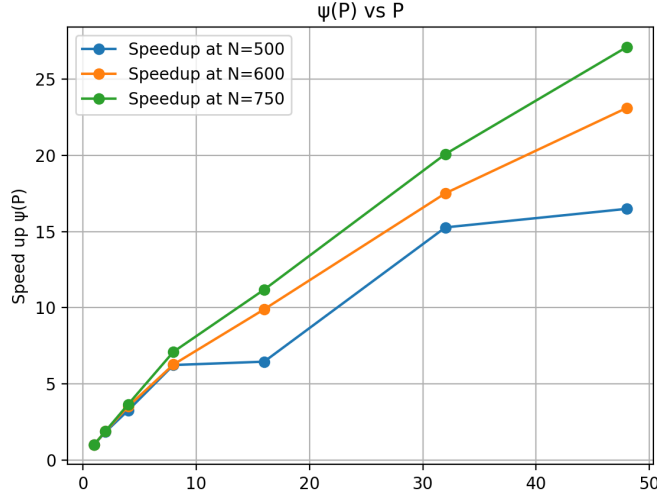


Figure 6: Speedup vs Processor count plot

$N = 600$, and $\psi \approx 27$ for $N = 750$. This behaviour is expected, as the gravitational force calculation scales as $\mathcal{O}(N^2)$, increasing the ratio of computation to communication as N grows.

At smaller processor counts, the speedup deviates from ideal linear scaling. This is particularly visible at intermediate values of P , where communication overheads, OpenMP thread management, and MPI synchronisation costs become comparable to the computational workload. Nevertheless, the overall trend demonstrates good strong-scaling performance for the system sizes considered.

3.2 Karp-Flatt Metric and Strong Scaling Limits

The corresponding Karp-Flatt metrics are shown in Figure 7. For all three values of N , the estimated serial fraction generally decreases as P increases, indicating that the parallel efficiency improves with increased computational load.

For $N = 500$, the Karp-Flatt metric remains at the level of a few percent for most values of P , suggesting that parallel overheads and residual serial regions impose a moderate but non-negligible limit on scalability. For $N = 600$ and $N = 750$, the estimated serial fraction is consistently lower, reflecting improved parallel efficiency as the problem size increases.

Some non-monotonic behaviour is observed in the Karp-Flatt metric at intermediate processor counts. This can be attributed to changes in the balance between computation and communication, as well as to differences in

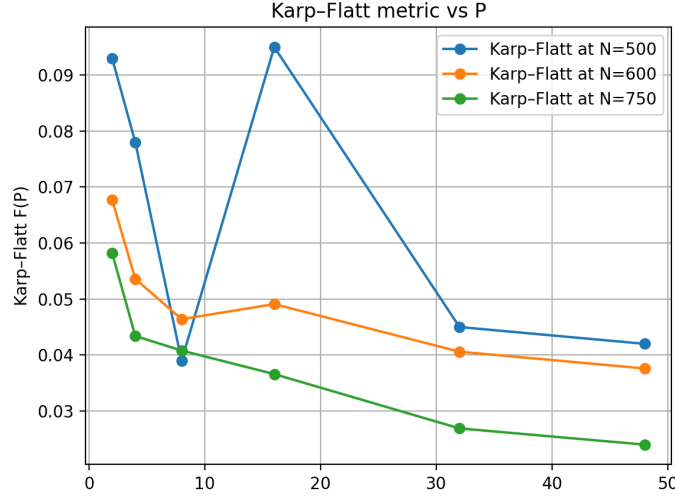


Figure 7: Karp-Flatt vs Processor count plot

MPI/OpenMP decomposition (more in the next section).

3.3 Effect of Hybrid MPI/OpenMP Decomposition

While Figure 6 shows the general trend in speedup, it loses information about hybrid decomposition and therefore individual performances of MPI and OpenMP, as it averages the simulation time per P .

To further investigate the hybrid parallel strategy, additional measurements were performed at fixed total processor counts ($P = 16$ and $P = 48$) while varying the decomposition between MPI tasks and OpenMP threads. The resulting speedup and Karp-Flatt metrics are shown in Figure 8.

Perhaps the performance is more easily seen in Figures 9 and 10, and Tables 3.3, 3.3 and 3.3 in which the P values are kept constant while varying MPI ranks and OpenMP threads.

At $P = 16$, configurations with a more balanced decomposition (e.g. 4×4 MPI $\times$ OpenMP) generally outperform highly asymmetric decompositions. This reflects a trade-off between MPI communication costs and OpenMP thread management overheads.

At $P = 48$, the optimal decomposition depends on system size. For larger N , configurations with fewer MPI ranks and more OpenMP threads tend to yield lower Karp-Flatt metrics, indicating reduced communication overhead. Conversely, for smaller N , excessive threading can introduce diminishing returns due to thread synchronisation costs.

Generally, it can be seen that having a larger number of MPI ranks at large

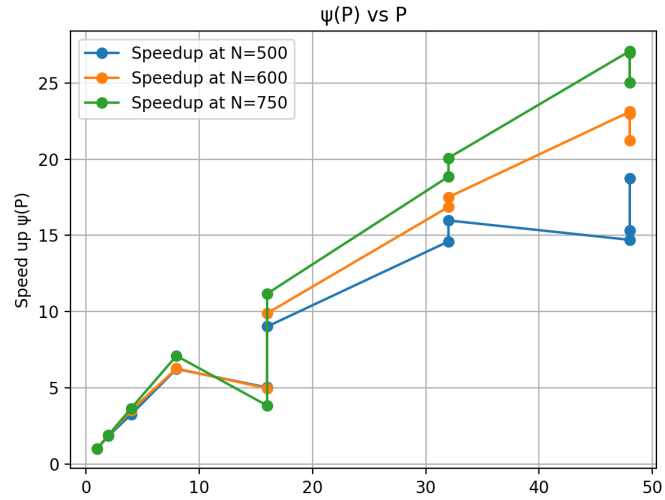
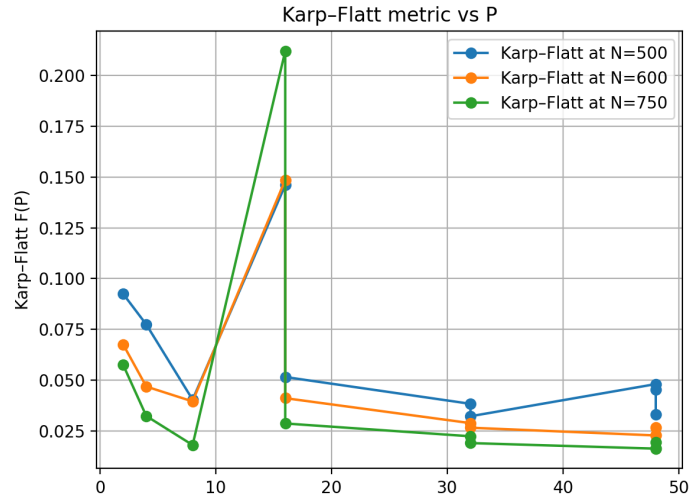


Figure 8: Karp-Flatt and Speed up vs Processor count plot (including hybrid decomposition performance)

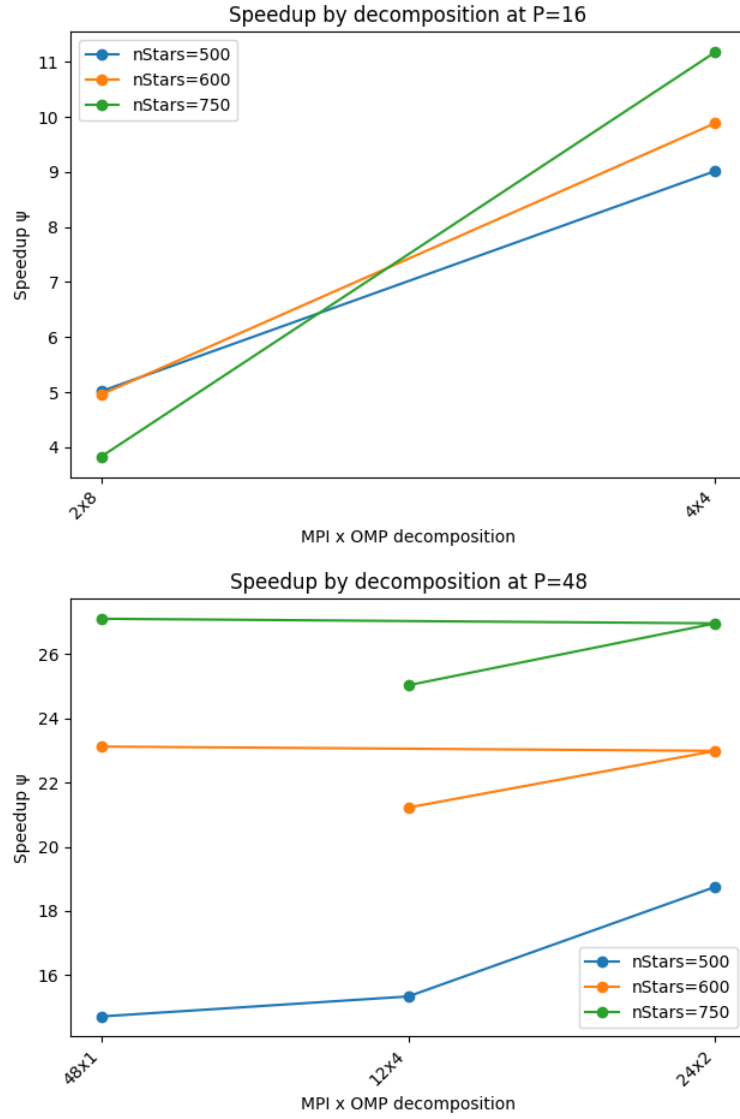


Figure 9: Speed up at constant P, demonstrating hybrid decomposition performance

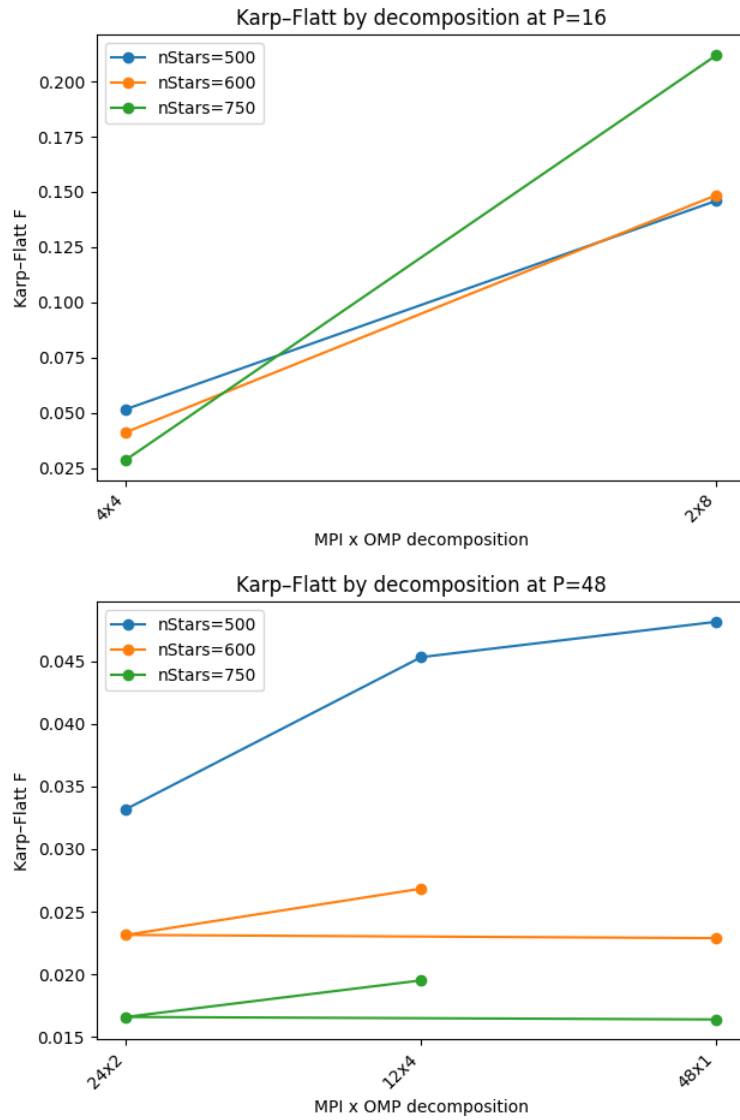


Figure 10: Karp-Flatt at constant P, demonstrating hybrid decomposition performance

N results in better performance.

MPI	OMP	P	Time (s)	Speedup ψ	Karp–Flatt F
1	1	1	30.1859	1.0000	–
1	2	2	16.4893	1.8306	0.0925
2	2	4	9.2998	3.2459	0.0774
4	2	8	4.8427	6.2332	0.0405
2	8	16	6.0188	5.0153	0.1460
4	4	16	3.3477	9.0169	0.0516
8	4	32	2.0671	14.6027	0.0384
16	2	32	1.8886	15.9836	0.0323
48	1	48	2.0519	14.7110	0.0481
12	4	48	1.9685	15.3345	0.0453
24	2	48	1.6098	18.7509	0.0332

Table 2: Performance results for `nStars` = 500.

MPI	OMP	P	Time (s)	Speedup ψ	Karp–Flatt F
1	1	1	43.3380	1.0000	–
1	2	2	23.1348	1.8733	0.0676
2	2	4	12.3599	3.5064	0.0469
4	2	8	6.9198	6.2629	0.0396
2	8	16	8.7438	4.9565	0.1485
4	4	16	4.3834	9.8868	0.0412
8	4	32	2.5691	16.8690	0.0289
16	2	32	2.4747	17.5125	0.0267
48	1	48	1.8741	23.1245	0.0229
12	4	48	2.0419	21.2239	0.0268
24	2	48	1.8851	22.9902	0.0231

Table 3: Performance results for `nStars` = 600.

3.4 Discussion and Limitations

Overall, the performance results indicate that the strong scaling of the code is primarily limited by parallel overheads rather than by large un-parallelised serial regions. The dominant $\mathcal{O}(N^2)$ force calculation parallelises effectively, while communication and synchronisation costs become increasingly significant at high processor counts.

The observed deviations from ideal scaling are therefore expected and reflect fundamental limitations of the algorithm and execution model rather than implementation errors. Given these constraints, the achieved performance represents a reasonable and efficient use of the available parallel resources.

MPI	OMP	P	Time (s)	Speedup ψ	Karp–Flatt F
1	1	1	67.6058	1.0000	–
1	2	2	35.7501	1.8911	0.0576
2	2	4	18.5464	3.6452	0.0324
4	2	8	9.5260	7.0969	0.0182
2	8	16	17.6573	3.8288	0.2119
4	4	16	6.0490	11.1763	0.0288
8	4	32	3.5843	18.8616	0.0225
16	2	32	3.3655	20.0878	0.0191
48	1	48	2.4939	27.1086	0.0164
12	4	48	2.7005	25.0348	0.0195
24	2	48	2.5071	26.9659	0.0166

Table 4: Performance results for `nStars` = 750.

While further optimisation—such as reducing communication frequency or employing more advanced domain decomposition strategies—could potentially improve scalability, such changes would require significant restructuring of the code and were deliberately avoided in order to preserve readability and correctness.

In summary, the measured speedup and Karp–Flatt metrics demonstrate that the chosen hybrid MPI/OpenMP parallelisation strategy successfully accelerates the simulation, particularly for larger system sizes. The results are consistent with theoretical expectations and provide clear evidence that the parallel implementation is both correct and effective within the constraints of the problem.

4 Final Full-Scale Simulation and Limitations

Finally, a full-scale simulation with `nStars`= 1500 and 100,000 timesteps was performed using the final parallel implementation at $P = 48$ and $P = 96$. $P = 48$ yielded a simulation time of 50.879018 *s*, whereas $P = 96$ yielded 27.534507 *s*. Unfortunately, I was only able to perform the final simulation at $P = 96$ and could not do other sections of the assignment, such as performance measurements, at this P due to lack of resource availability. The job took several minutes to execute as the resources weren’t available. I ultimately chose to restrict my performance measurements up to $P = 48$ for the sake of brevity, time and resource availability. However, the final run satisfies all of the assignment’s requirements and produces a promising result, which is well under 1 minute.